

Checklist

Observability Implementation

1. Observability Strategy

- Define observability as a first-class platform capability, not an afterthought.
- Identify the business and engineering outcomes observability should support.
- Define how metrics, logs, and traces will be collected and correlated.
- Establish a Single Pane of Glass (SPOG) strategy rather than creating disconnected dashboards.
- Identify measurable outcomes such as reduced MTTD and MTTR.

2. Metrics, Logs, and Traces

- Collect **metrics** to understand what happened.
- Collect **logs** to understand how it happened.
- Collect **traces** to understand why it happened.
- Use common identifiers such as service names and trace IDs to correlate telemetry.
- Ensure telemetry covers both applications and supporting infrastructure.

3. OpenTelemetry Standardization

- Instrument applications using OpenTelemetry SDKs.
- Deploy OpenTelemetry collectors.
- Adopt consistent semantic conventions and attribute names.
- Export telemetry using vendor-neutral OTLP.
- Keep application instrumentation independent of the observability backend.
- Configure collectors for batching, buffering, retries, and high availability.

4. Metrics Infrastructure

- Deploy a time-series database such as Prometheus.
- Expose application metrics through appropriate `/metrics` endpoints.
- Enable automatic service discovery for metrics targets.
- Define appropriate data-retention policies.
- Configure backups and scaling for observability databases.
- Track metrics that reveal degradation before a complete failure occurs.

5. Telemetry Ingestion

- Use a **pull model** where appropriate for metrics.
- Use a **push model** for distributed traces.
- Use an appropriate hybrid approach for logs.
- Verify that telemetry can cross required network boundaries.
- Ensure telemetry ingestion does not create excessive load on application workloads.

6. Platform and Developer Responsibilities

- Make the platform team responsible for shared observability infrastructure.
- Make development teams responsible for instrumenting their applications.
- Provide reusable APIs, SDK configurations, and starter kits.
- Provide self-service observability capabilities through the platform.
- Establish clear observability contracts and non-functional requirements.

7. Build vs. Buy

- Assess the organization's maturity, service count, budget, and staffing.
- Evaluate commercial SaaS for small or early-stage environments.
- Consider hybrid open-source/commercial approaches as the organization grows.
- Evaluate self-hosted solutions where scale, customization, or economics justify them.
- Account for compliance, data-residency, and audit requirements.
- Revisit the build-versus-buy decision as organizational maturity changes.

8. Observability Personas

- Identify the primary consumers of observability data.
- Provide service-health information for external customers.
- Provide business-level KPIs for executives.

- Provide detailed debugging views for developers.
- Provide environment and release-validation views for QA.
- Provide incident-diagnostic capabilities for DevOps and SRE teams.
- Provide forensic and authentication-related information for security teams.
- Provide immutable audit information for compliance and governance teams.

9. Dashboards

- Provide preconfigured dashboards for common personas.
- Use a consistent visualization layer such as Grafana.
- Connect dashboards to multiple telemetry backends where required.
- Store dashboard definitions in version control.
- Create reusable dashboard templates with variables.
- Display deployment, incident, and configuration-change markers on dashboards.
- Restrict dashboard editing to appropriate owners.
- Provide view-only access where modification is unnecessary.
- Review and optimize slow dashboard queries.

10. Security Observability

- Expose policy violations as observability metrics.
- Track RBAC violations and network-policy blocks.
- Track container vulnerabilities by severity.
- Surface sanitized aggregate security metrics broadly.
- Restrict sensitive security details to authorized users.
- Define measurable security SLOs where appropriate.
- Connect security events to distributed traces for investigation.

11. CI/CD Integration

- Make observability validation part of CI/CD.
- Fail pipelines when required metrics endpoints are missing.
- Fail pipelines when required structured logs are not emitted.
- Fail pipelines when required trace spans are absent.
- Verify telemetry in staging before promotion to production.
- Publish deployment metadata as telemetry.
- Correlate deployments automatically with subsequent telemetry.
- Make CI/CD pipeline performance observable.

12. Pipeline Observability

- Measure build duration.
- Measure test execution time.
- Track deployment frequency.
- Measure mean time to build.
- Track test flakiness.
- Identify and optimize CI/CD bottlenecks.

13. Alerting

- Alert primarily on user-impacting symptoms rather than internal causes.
- Ensure every alert clearly answers: **What action should I take?**
- Include severity and runbook information with alerts.
- Use SLO-based alerting where appropriate.
- Implement fast-burn alerts for severe outages.
- Implement slow-burn alerts for gradual degradation.
- Group related alerts to reduce noise.
- Route alerts according to business impact.
- Silence alerts during known maintenance windows.
- Consider rate-of-change alerts instead of relying only on absolute thresholds.
- Review and prune alerts regularly.
- Remove or redesign alerts that repeatedly fire without producing action.

14. SLIs, SLOs, and Error Budgets

- Define SLIs that reflect actual user experience.
- Define measurable SLO targets for important services.
- Establish error budgets.
- Publish SLO status on observability dashboards.
- Display real-time error-budget consumption.
- Use error-budget health to guide deployment velocity.
- Define SLOs for internal platform services as well as customer-facing applications.

15. SLOs as Code

- Store SLO definitions in Git alongside application code.
- Review SLO changes using the normal code-review process.

- Make SLO definitions rollback-capable.
- Deploy SLO definitions automatically through GitOps.
- Synchronize recording rules, dashboards, and alert configuration.
- Validate deployments against SLOs before production promotion.
- Provide reusable SLO templates or CRDs for development teams.
- Track SLO changes in Git for audit purposes.
- Use consistent SLO definitions across environments while allowing appropriate threshold differences.

16. Deployment Lifecycle

- Deploy the application.
- Generate deployment metadata.
- Generate telemetry.
- Correlate deployment events with metrics, logs, and traces.
- Detect SLO violations.
- Generate an actionable alert.
- Notify the responsible team.
- Trigger rollback or remediation when appropriate.

Final Readiness Check

- Metrics, logs, and traces are available and correlated.
- OpenTelemetry instrumentation is standardized.
- Telemetry collection is automatic.
- Dashboards support the required personas.
- Security telemetry is incorporated.
- CI/CD prevents uninstrumented code from reaching production.
- Alerts are actionable and low-noise.
- SLIs, SLOs, and error budgets are defined.
- SLOs and dashboards are version controlled.
- Deployment events can be correlated with incidents.
- The platform can demonstrate measurable improvements in reliability and MTTR.